

PLAN CADRE

Titre du cours : Programmation graphique
Discipline : Techniques de l'informatique

Code du cours : 420-J30-BB
Programme : Programmeur de jeux vidéo
(LEA.C7)

Préalables : 201-J25-BB, Algèbre vectorielle appliquée

Préparatoire à : Aucun

Durée : 45h

Pondération : 2-1-3
Unités : 2

Code de la compétence	Compétence	Entièrement	Partiellement
BAYT	Décrire les concepts et les mécaniques du système de rendu dans le jeu	X	

Éléments de compétence(s) concerné(s) :

BAYT-1. Expliquer les mécaniques du système de rendu.
BAYT-2. Écrire des algorithmes simples de rendu.
BAYT-3. Décrire le processus du traitement des données graphiques par les algorithmes.
BAYT-4. Utiliser l'interface de programmation (API) d'un engin graphique.

NOTE PRÉLIMINAIRE

Contribution du cours au programme de formation

Le cours 420-J30-BB (*Programmation graphique*) se donne à la troisième session du programme « Programmeur de jeux vidéo ». Il a comme préalable le cours 201-J25-BB, Algèbre vectorielle appliquée. Ce cours permettra à l'étudiant d'expliquer les mécaniques du système de rendu. D'écrire des algorithmes simples de rendu. De d'écrire le processus du traitement des données graphiques par les algorithmes. De plus, les étudiants apprendront à utiliser l'interface de programmation (API) d'un engin graphique.

Orientations pédagogiques

Le cours 420-J30-BB (*Programmation graphique*) vise à donner à l'étudiant une vue d'ensemble sur le fonctionnement d'un engin de rendu ainsi que des outils qui y sont rattachés (outil de modélisation 3D et outil de production et de modification d'images), et également une compréhension des techniques principales d'éclairage, d'ombrage et de texturisation présentes dans un pipeline de rendu de jeu. Il vise aussi à donner une vue d'ensemble de l'utilisation d'une interface de programmation (API) d'un engin graphique.

Le cours est donné en laboratoire informatique, sous forme de cours magistraux tout en privilégiant principalement l'aspect pratique : travaux dirigés et travaux pratiques.

PARAMÈTRES PÉDAGOGIQUES

Énoncé de compétences : Décrire les concepts et les mécaniques du système de rendu dans le jeu

Cours : Programmation graphique

OBJECTIFS TERMINAUX	ÉLÉMENTS DE COMPÉTENCE	BALISES DE CONTENU	CRITÈRES D'ÉVALUATION
Reconnaître le vocabulaire relié à la programmation graphique. Décrire les différentes transformations mathématiques nécessaires au rendu d'un objet 3D. Décrire la représentation des composantes d'un objet 3D. Identifier les différentes ressources matérielles utilisées lors de rendu. Décrire sommairement les différentes rendu présentes dans un API graphique la description des primitives à l'affich	1, 2	Notions mathématiques : • Vecteurs • Matrices • Systèmes de coordonnées hiérarchiques (local/monde/objet) <i>Ces notions devraient être connues par l'étudiant car elles sont enseignées dans le cours d'algèbre vectorielle appliquée. Seul un court rappel de la matière devrait être nécessaire.</i> Description de la géométrie : • Vertex ○ Position ○ UV ○ Normale ○ Tangente ○ Binormale ○ Couleur • Types de primitives • Orientation des vertex	<i>Évaluation formative</i> <i>Évaluation sommative</i> 1.1 Identification juste des ressources matérielles utilisées lors de rendu. 1.2 Description appropriée des données de rendu. 1.3 Utilisation d'une terminologie appropriée. 1.4 Détermination complète de la représentation des composantes d'un objet 3D (vertex, face, textures, UV, etc.). 1.5 Interprétation fonctionnelle de la représentation d'objets complexes (table, personnage, maison, arbre, etc.). 1.6 Description précise du concept de caméra. 1.7 Transcription concrète de la représentation de scènes. 1.8 Description des pipelines de rendus (Forward, Deferred, etc.). 1.9 Description juste des effets des sources de lumière. 1.10 Identification précise des principaux tampons d'un pipeline de rendu. 1.11 Description juste du processus de post-FX.

		Ressources graphiques : Texture (2D, Cube, Volume) • Pixel shader • Vertex shader • Geometry shader • Vertex buffer • Index buffer • Render targets • Render states Systèmes de coordonnées : • Modèle • Monde • Caméra • Écran Projections : • Orthographique • Perspective Étapes simplifiées du pipeline de rendu: • Données d'entrées et assemblage des primitives • Vertex shader • Geometry shader • Flot de sortie • Rastérisation • Pixel shader • Test de profondeur et de stencil • Blending • Masquage et écriture du pixel	2.1 Description détaillée des algorithmes permettant la modification des objets 3D (mise à l'échelle, rotation, translation, etc.). 2.2 Description explicite des algorithmes de projection. 2.3 Écrire des algorithmes simples de rendu. 2.4 Écriture correcte d'un algorithme simple de rendu.
--	--	--	--

		Pour ne pas trop complexifier l'apprentissage du pipeline de rendu, il vaudrait mieux se limiter au pipeline de niveau DirectX 10.	
Décrire les techniques principales d'éclairage dynamique et statique. Décrire les techniques principales d'ombrage dynamique. Décrire les techniques principales de texturisation. Identifier les différentes utilisations des images dans un contexte de jeu.	3.	Techniques générales de rendu : - Rendu direct (forward rendering) - Rendu différé (deferred rendering) <i>Le but ici est de comprendre la différence entre les deux types de techniques, ainsi que les avantages et inconvénients de chacune.</i> Techniques simples d'application de textures dans un engine de rendu. • Opaque • Alpha blending • Alpha test • Texture en tant que masque <i>Il serait recommandé d'utiliser un pipeline de jeu professionnel tel UDK ou Unity pour permettre à l'étudiant de rapidement voir l'intégration de ses images dans un engine de rendu. Il pourra y créer des matériaux simples utilisant les images qu'il aura créées.</i>	3.1 Description adéquate de différentes techniques de rendu. 3.2 Description adéquate de techniques d'application de textures. 3.3 Description adéquate de techniques d'éclairage et d'ombrage. 3.4 Description adéquate de techniques avancées de texturisation et de filtrage (bump mapping, shaders, PBR, etc.).

		Survol de l'équation de BRDF (Bidirectional Reflectance Distribution Function). <i>Le but ici n'est pas de rentrer dans les détails des différentes équations de BRDF, mais de comprendre la base de la théorie.</i> Modèles d'éclairage: - Plat - Gouraud - Phong - Blinn-Phong Types de sources de lumière : - Spot - Directionnelle - Omni Types d'éclairage : - Ambient - Diffus - Spéculaire - Émissif Effets de lumière : - Fresnel Autre type de lumière : - Projection de texture Techniques d'éclairage statique : - Lightmaps - Couleurs au vertex	
--	--	---	--

		Terminologie reliée à l'ombrage dynamique : - Source de lumière Obturbateur - Récepteur - Ombre - Pénombre Techniques d'ombrage dynamique : - Ombres projetées - Ombres volumétriques (shadow volumes) - Carte d'ombrage (shadow map) <i>L'idée ici n'est pas de voir toutes les techniques en détail, mais bien d'acquies une connaissance des différentes catégories de techniques d'ombrage dynamique.</i> Les différents types de texture : - 1D - 2D - 3D - Cube map Techniques de texturisation : - Normal mapping - Parallax mapping - Relief mapping - Diffuse map - Specular map - Gloss map Techniques d'animation de texture : - Changement dynamique de texture	
--	--	--	--

		- Animation de matrice de texture - Animation d'UV de géométrie Correction gamma. Structures spatiales de données : - Volumes englobant hiérarchiques - Arbre BSP - Octree - Quadtree - Graphe de scène (scene graph) Techniques de suppression d'éléments graphiques indésirables (culling) : - Pyramide tronquée (frustum) - Occlusion - Occlusion GPU - Niveaux de détails (LOD) - Portal	
--	--	--	--

Identifier l'implémentation des données dans une interface de programmation (API) d'un engin graphique. Identifier les différents services fournis par l'interface de programmation (API) d'un engin graphique. Reconnaître les éléments d'un engin de rendu pouvant affecter la performance du rendu. Employer une interface de programmation (API) d'un engin graphique. Employer des outils de validation d'engin de rendu.	4.	<i>Pour cette section, il est recommandé d'utiliser une interface de programmation (API) d'un engin graphique telle Ogre3D. Tout le code source sera alors disponible pour faciliter l'apprentissage. De plus, Ogre3D possède une bonne documentation et supporte beaucoup de techniques mentionnées dans la section précédente.</i> Représentation des données dans l'interface de programmation choisie: - Géométrie statique - Géométrie dynamique - Matériel - o États de rendu - o Shaders - Texture - Scène Services de rendu bas niveau: - Render target - Fenêtre d'affichage (viewport) - Caméra - Lumière Services de rendu haut niveau : - Affichage à tête haute (head-up-display) - Skybox - Projection de texture - Ombrage dynamique Billboard	4.1 Identification correcte de l'implémentation des données dans une interface de programmation (API) spécialisée. 4.2 Identification adéquate des différents services fournis par l'interface de programmation (API) spécialisée. 4.3 Codification logique du modèle de classes en utilisant l'interface de programmation (API) spécialisée. 4.4 Mise à profit des fonctionnalités d'accélération du système. 4.5 Validation judicieuse de l'engin de rendu.
--	-----------	--	--

ÉVALUATION FINALE

Pondération : 45%

Formulation de la performance finale attendue de l'élève au terme de ce cours

À la fin de ce cours, l'étudiant devra être capable d'énumérer et d'expliquer les différentes techniques et les mécaniques du système de rendu dans le jeu en liens avec des techniques principales d'éclairage, d'ombrage et de texturisation présentes dans un pipeline de rendu de jeu. Il sera en mesure d'employer une interface de programmation (API) d'un engin graphique afin d'implémenter des techniques graphiques.

Forme de l'épreuve terminale

- Des questions sur les concepts vus en classe. (partie théorique)
- La réalisation de programmes ou de sections de programmes en utilisant l'interface de programmation (API) d'un engin graphique choisie pour le cours.(partie pratique)

Contexte de réalisation de l'épreuve terminale

- Individuellement dans un laboratoire d'informatique.
- Durée minimale de 3 heures
- Dans la partie pratique, les étudiants peuvent utiliser une documentation autorisée par le professeur.

Contexte de réalisation pour la partie pratique :

- Individuellement.
- À partir de spécifications établies par l'enseignant.
- En utilisant l'environnement de développement et l'interface de programmation (API) d'un engin graphique choisie pour le cours.

Critères d'évaluation propres à l'épreuve terminale

- Respect des consignes.
- Justesse des explications. (partie théorique)
- Fonctionnalité des programmes. (partie pratique)
- Utilisation appropriée des fonctionnalités d'une interface de programmation (API) d'un engin graphique. (partie pratique)

MÉDIAGRAPHIE DE BASE

Médiagraphie pour les élèves :

- Tomas Akenine-Moller, Eric Haines, Naty Hoffman. Real-Time Rendering, Third Edition. A K Peters/CRC Press, 3 edition, 2008.

Médiagraphie pour les enseignants :

- Eric Lengyel. Mathematics for 3D Game Programming and Computer Graphics. Course Technology PTR, 3 edition, 2011.
- Ben Garney and Eric Preisz. Video Game Optimization. Course Technology PTR, 1 edition, 2010.

LOGICIELS

Microsoft Studio Visual

C++3DS Max ou Maya

Photoshop ou Gimp (*Photoshop est probablement le logiciel le plus utilisé dans l'industrie*)

Unity ou UDK

Ogre3D (*ou l'interface de programmation d'engin graphique sélectionnée pour le cours*)

DirectX10 SDK